

VEŠTAČKA INTELIGENCIJA, 2010/2011

REŠAVANJE PROBLEMA I TRAŽENJE

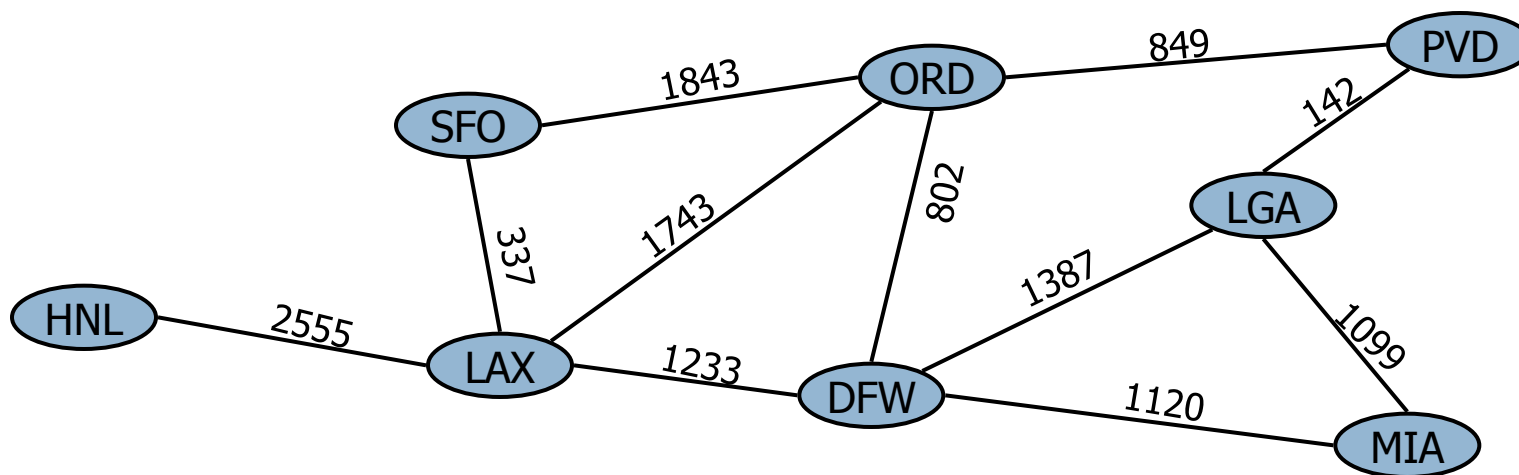
Sadržaj

- Formulacija problema
- Slepí algoritmi za traženje
- Informisani algoritmi za traženje
- Algoritmi za igre



Podsetnik: Graf

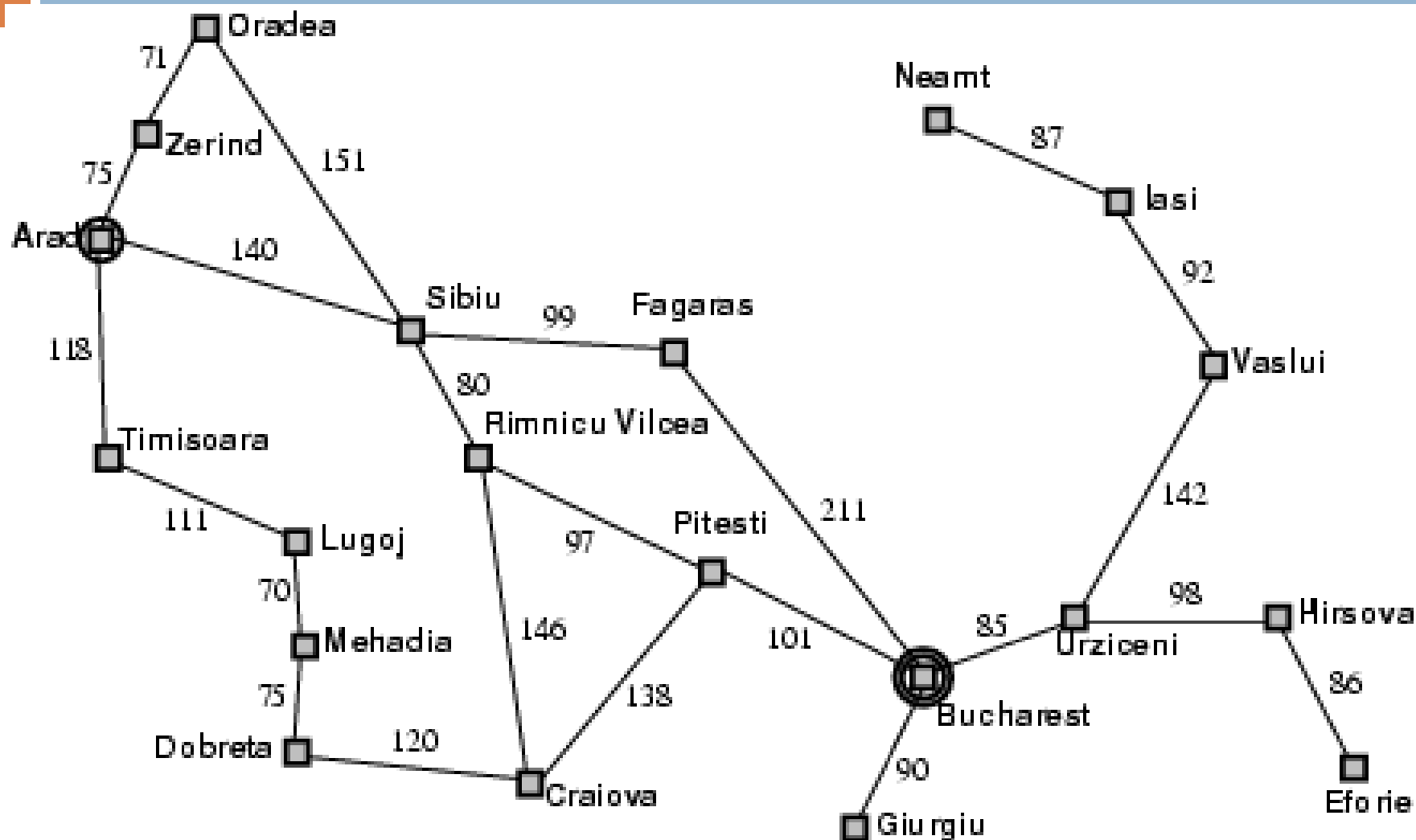
- Graf je uređeni par (V, E) , gde je
 - V skup temena v , koji se zovu i **čvorovi**
 - E je kolekcija parova čvorova $e=[u,v]$, koji čine **potege** ili **grane**
 - Čvorovi i grane čuvaju odgovarajuće vrednosti (graf je **obeležen**)
 - Svakom potegu u grafu može biti pridružen broj i takav graf se naziva **težinski graf** ili **mreža**.
 - Nenegativan broj pridružen potegu naziva se **težina**.



Primer: Rumunija

Kako naći put od Arada do Bukurešta?

3



Problem-solving agenti

4

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

static: *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then do**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

seq $\leftarrow$ SEARCH(*problem*)

action $\leftarrow$ FIRST(*seq*)

seq $\leftarrow$ REST(*seq*)

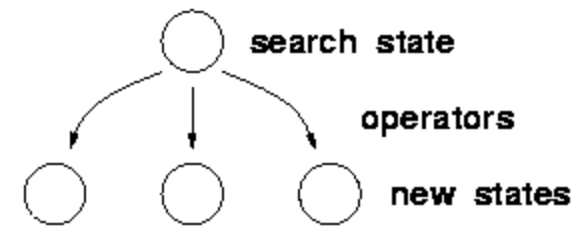
return *action*

Tipični problemi

- traženje puta (route finding)
- Najkraci put koji obilazi zadata mesta (travelling salesperson)
- VLSI layout
- Navigacija za robote
- Planiranje letova za avio kompaniju
- ...

Rešavanje problema kao traženje

- **Stanje** – diskretno stanje «sveta»
- **Prostor stanja**: - skup svih stanja problema
- **Početno stanje**
- **Ciljno stanje** (ili stanja!!)
- **Operatori**: - prelaz iz stanja u stanje; funkcije koje određuju sledeće stanje na osnovu tekućeg stanja (ili NIL)
- **Sledbenici** (potomci): sva stanja u koja se može preći iz tekućeg stanja
- **Traženje sledbenika**: primena svih operatora na tekuće stanje da bi se dobili potomci

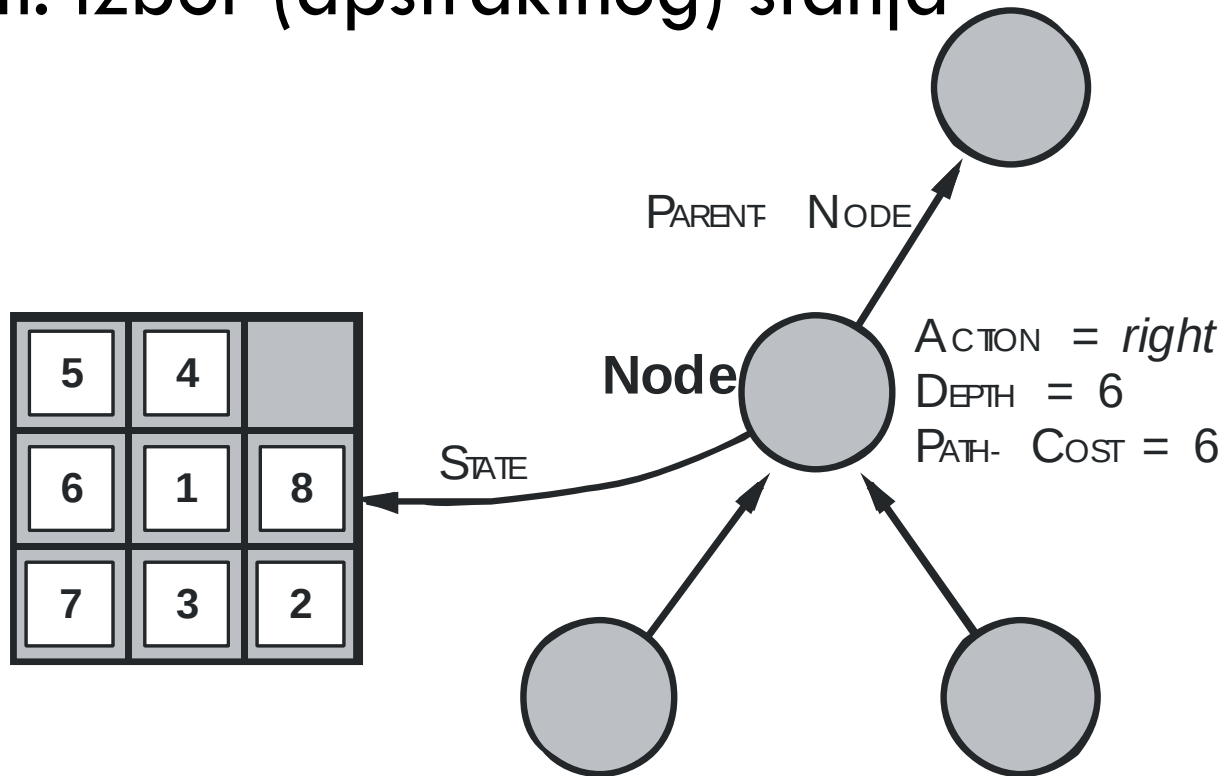


Rešavanje problema kao traženje

- **Test**: predikat koji vraća T ili F za zadato stanje
- **Cena puta** (opciono): suma cena individualnih operatora
- **Heuristika** – informacija o tome zbog čega je izbor nekog čvora (stanja) bolji od izbora ostalih
- **Rešavanje problema** – odrediti niz operatora koji prevodi sistem iz početnog stanja u ciljno stanje; put do cilja (**rešenje problema**) je niz čvorova u grafu koje treba obići da bi se došlo do cilja.

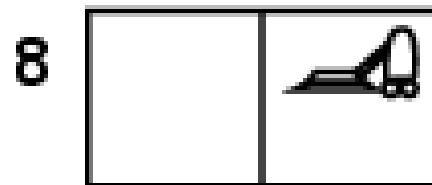
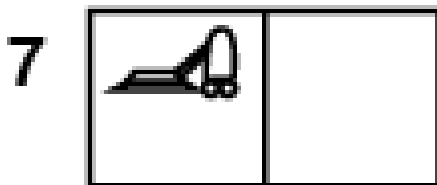
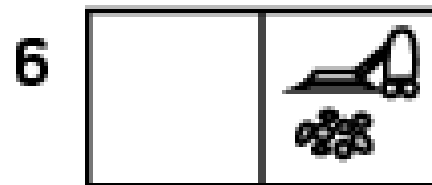
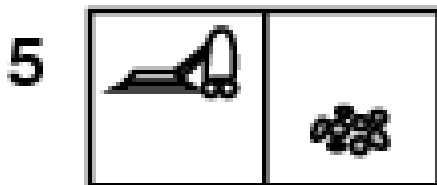
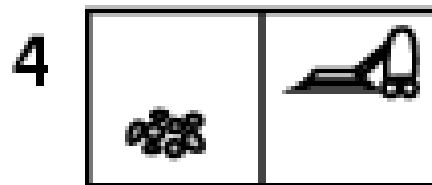
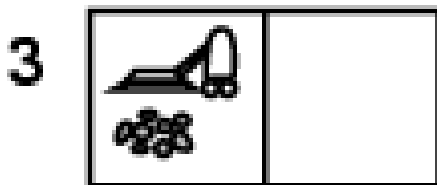
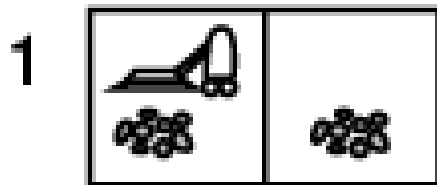
Stanje problema : čvor u prostoru

Problem: izbor (apstraktnog) stanja



Primer: vacuum world

Moguća stanja i operatori



Operatori:

L – LEVO

R – DESNO

S – USISAJ

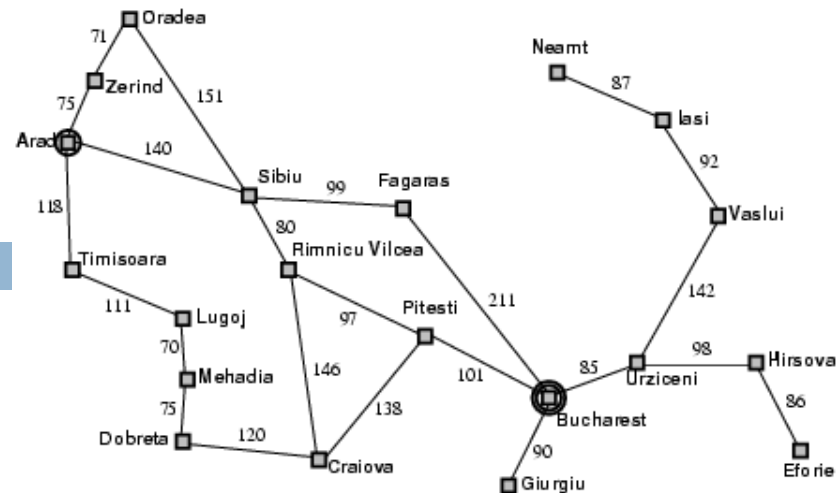
Formulacija problema

- **Kritična i** najvažnija faza
- Biraju se važne osobine problema
- One se predstavljaju prostorom problema / stanjima

- **Koraci** formalnog predstavljanja problema u veštačkoj inteligenciji
 - ▣ **Definicija problema**, njegovog okruženja i prostora stanja.
 - ▣ **Definicija početnih stanja** unutar prostora stanja.
 - ▣ **Definicija ciljnih stanja** (u njima se prekida proces pretrage).
 - ▣ **Definicija skupa operatora** i pravila koja identifikuju uslove koji moraju biti zadovoljeni da bi se primenio odgovarajući operator.

Formulacija problema

11



Problem se definiše preko:

- **inicijalnog stanja** napr., “Arad”
- **akcija** ili **funkcije sledbenika** $S(x)$ = skup parova akcija–stanje
 - ▣ Primer: $S(\text{Arad}) = \{ \langle \text{Arad} \rightarrow \text{Zerind}, \text{Zerind} \rangle, \dots \}$
- **test na ciljno stanje**,
primer: $x = \text{“Bucharest”}$, Čisto, $\text{ŠahMat}(x), \dots$
- **cena puta**
 - ▣ Primer: zbir rastojanja, broj izvršenih akcija itd.
 - ▣ $c(x, a, y)$ je **cena koraka**, uz pretpostavku ≥ 0
- **rešenje** je sekvenca akcija počev od inicijalnog stanja do ciljnog stanja

Primer: 8-puzzle, formulacija 1

- Prostor stanja
 - Svako stanje je polje 3x3 sa brojevima
 - Lokacija pločica
- Početno stanje: stanje 1
- Ciljno stanje: stanje 2
- Operatori:
 - levo, desno, gore, dole, koje se primenjuju na bilo koji element polja (ili NIL)
- Algoritam vraća niz operatora (put)
- Cena: 1 po potezu

5	4	
6	1	8
7	3	2

1	2	3
8		4
7	6	5

Problemi sa formulacijom 1

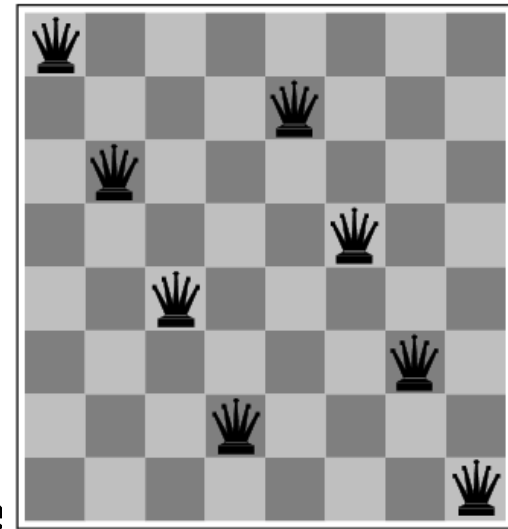
- **Koliko** operatora moze da se primeni na početno stanje?
- **Kako** se odredjuju sledbenici početnog stanja?
- Bolje je ako se **operatori mogu primeniti u većini slučajeva**

8-puzzle: formulacija 2

- Isti operatori, ali se primenjuju na **praznu poziciju**
- Operatori tada ne zahtevaju argumente
- Algoritam i dalje vraća putanju

Problem 8 kraljica: formulacija 1

- Prostor stanja:
 - ▣ polje 8x8, svaki element može sadržati $\leq n$ kraljica, pozicija otvoren/napadnut.
- Početno: prazna tabla
- Ciljno stanje: raspoređene kraljice
- Operatori:
 - ▣ 8 operatora (po jedan za svaki red)
 - ▣ Kraljica se smesta u prvo slobodno (nenapadnuto) polje

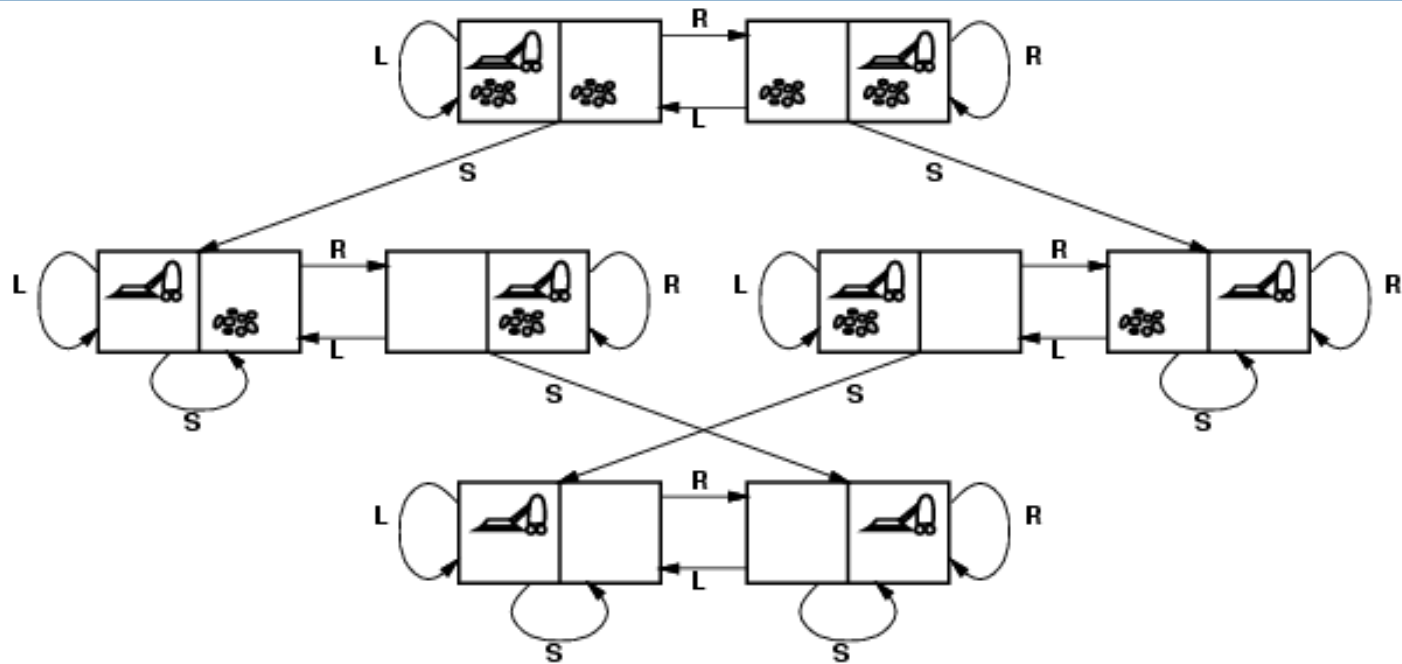


Problem 8 kraljica: formulacija 2

- Počni sa slučajno razmeštenim kraljicama
- Razmestati kraljice dok nema konflikata
- Stanje: vektor od 8 elemenata (vrste)
- Operatori: pomeraju napadnute kraljice u drugi red u istoj koloni

Primer: vacuum world

Prostor traženja



Stanja? integer: smeće i lokacija

Poč. stanje? Bilo koje

Akcije? Left, Right, Suck

Test ciljnog stanja? Nema smeća na svim lokacijama

Cena puta? 1 po akciji

Traženje

Šta radi algoritam za traženje?

- Polazno stanje
- Operatori
- Sledbenici
- Krajnje stanje (cilj)
- Test
- Cena puta

- **Obilazak prostora stanja** -
 - Pronalazi jednu putanju (ili sve putanje) od početnog do ciljnog stanja
 - Izračunava cenu aplikacije niza operatora (put) koji vodi od početnog stanja to krajnjeg stanja
 - Neki algoritmi nalaze put sa minimalnom cenom

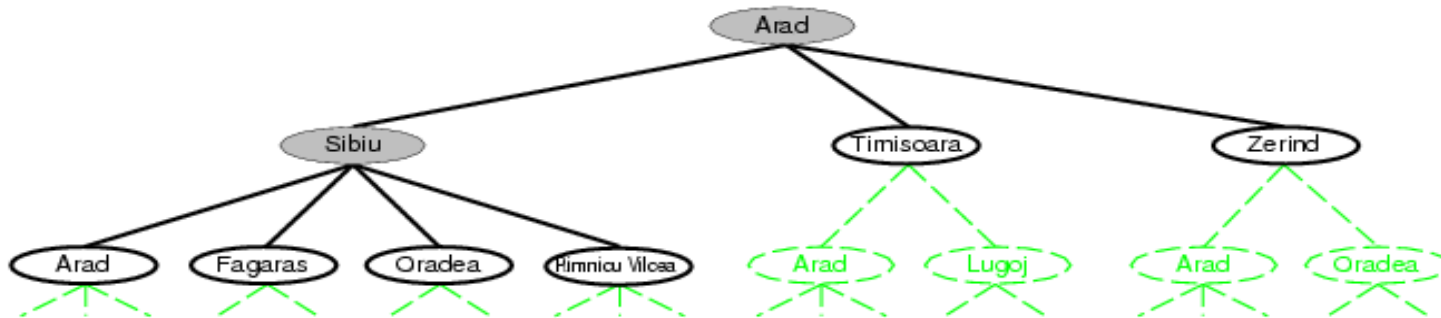
Stablo traženja

- Stablo traženja je efikasan način da se predstavi kako algoritam traženja ispituje prostor traženja.
 - ▣ Dinamički se kreira
 - ▣ Dubina stanja: razdaljina od korena
 - ▣ Faktor granjanja: prosečni broj sledbenika
 - ▣ Granica (frontier): otvoreni cvorovi (listovi)

Vazno: prostor traženja i stablo traženja se razlikuju!

Primer stabla traženja

20



```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
```

Opšti algoritam traženja

- **Ulaz:** prostor traženja sa

 - početnim stanjem S , i

 - ciljem G

- Algoritam:

S je koren stabla trazenja. Prvi element **pom.strukture** je S .

Petlja: ako ima kandidata za ekspanziju (red nije prazan)

Uzeti prvi cvor iz reda.

Ako je izabrani cvor jednak cilju, kraj

Naći sve sledbenike izabranog cvora, i dodati ih u
stablo (**prema zadatoj strategiji**)

Strategije

22

- **Strategija** je definisana izborom čvorova za ekspanziju
- Evaluacija strategija
 - ▣ Kompletnost: da li nalazi rešenje?
 - ▣ Cena – vreme i memorija
 - ▣ Optimalnost – da li nalazi rešenje koje najmanje košta
- Cena tj vremenska i prostorna (memorijska) kompleksnost se mogu meriti u zavisnosti od:
 - ▣ b : maksimalne vrednosti faktora grananja stabla traženja
 - ▣ d : dubine najjeftinijeg rešenja
 - ▣ m : maksimalna dubina prostora traženja (može biti ∞)

Osnovni algoritmi za traženje

- **Neinformisani (slepi) algoritmi**

(depth-first search, breadth-first search, ...)

Ne koriste nikakvo znanje o domenu

- **Informisani (heuristicki) algoritmi (A^*)**

Koriste heuristike da pronadju najbolje resenje

Neinformisani algoritmi traženja

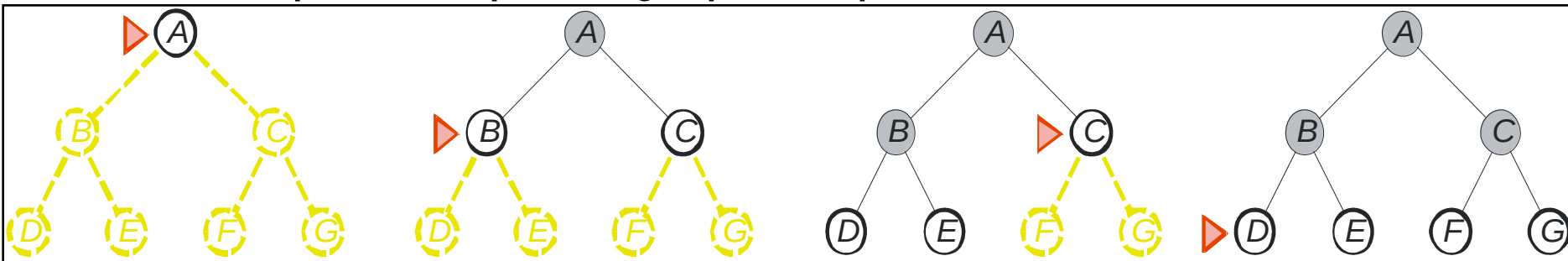
- Breadth-first search – traženje po širini
- Uniform-cost search – traženje sa uniformnom cenom
- Depth-first search – traženje po dubini
- Iterative deepening search – ograničavanje traženja po dubini

Traženje po širini

- *Breadth-first search*
- Čvorovi se obilaze s leva u desno
- Da bi se **izbegle petlje**, ne generisu se sledbenici koji su jednaki roditelju tekuceg cvora (postoje i drugi nacini da se izbegnu petlje)
- Nalazi najplići cilj
- Kompletan i optimalan ako je cena puta neopadajuca funkcija dubine čvora
- **Implementacija: red (FIFO)**

Algoritam traženja po širini

1. Formirati listu čvorova koja inicijalno sadrži samo startni čvor.
2. Dok se lista čvorova ne isprazni ili se ne dođe do ciljnog čvora, proveriti da li je prvi element liste ciljni čvor
 - a) Ako je prvi element liste ciljni čvor, ne raditi ništa.
 - b) Ako prvi element liste nije ciljni čvor, ukloniti ga iz liste i dodati sve njegove sledbenike iz stabla pretrage (ako ih ima i ako nisu već posećeni) na **kraj liste**.
3. Ako je pronađen ciljni čvor, pretraga je uspešno završena; u suprotnom pretraga je neuspešna.



Traženje po širini

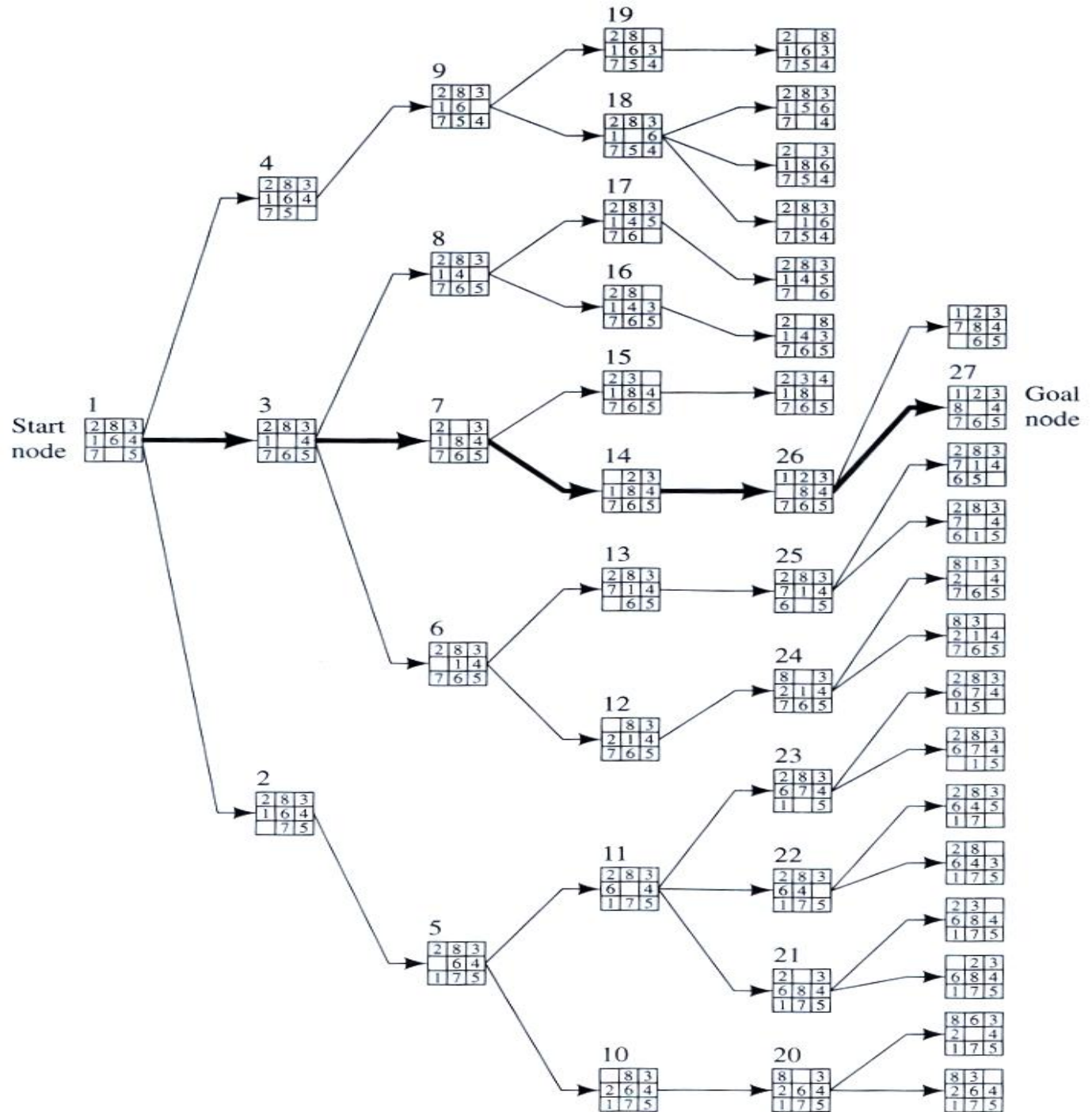
- Prednosti:

- Uvek nalazi cilj (rešenje)
- Dobar za plitka stabla

- Nedostaci:

- Veliki zahtevi za memorijom
- Može biti spor za stabla sa velikom dubinom

Primer BFS



Varijante:

Traženje sa uniformnom cenom

- Uniform cost search
- Modifikacija trazenja po širini
- Iz reda se bira čvor sa najnižom cenom $g(n)$
- Nalazi optimalno rešenje ako cena puta nikada ne opada sa dubinom

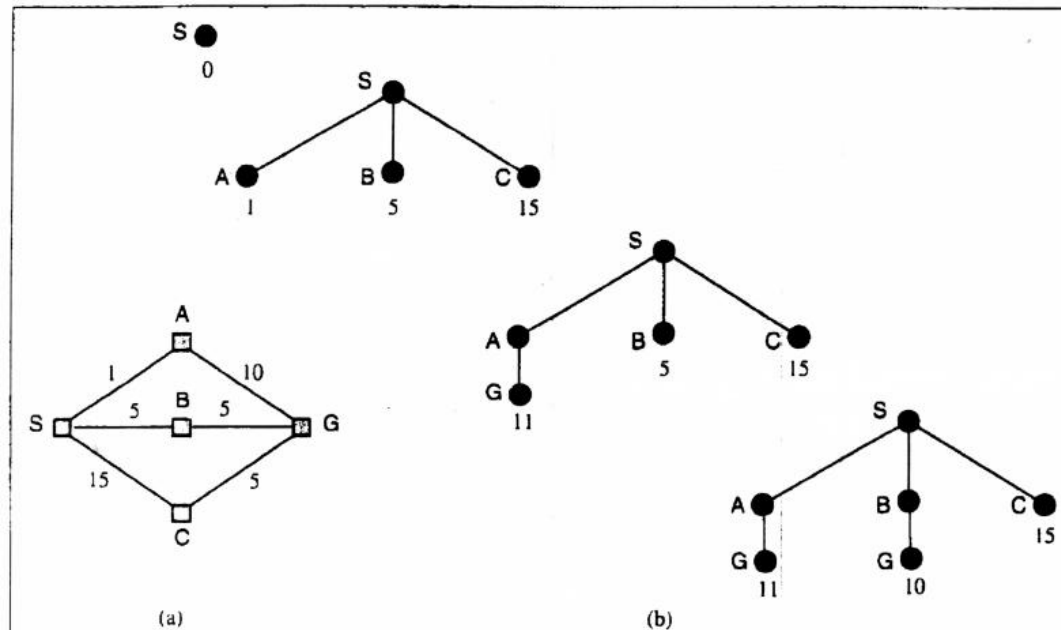
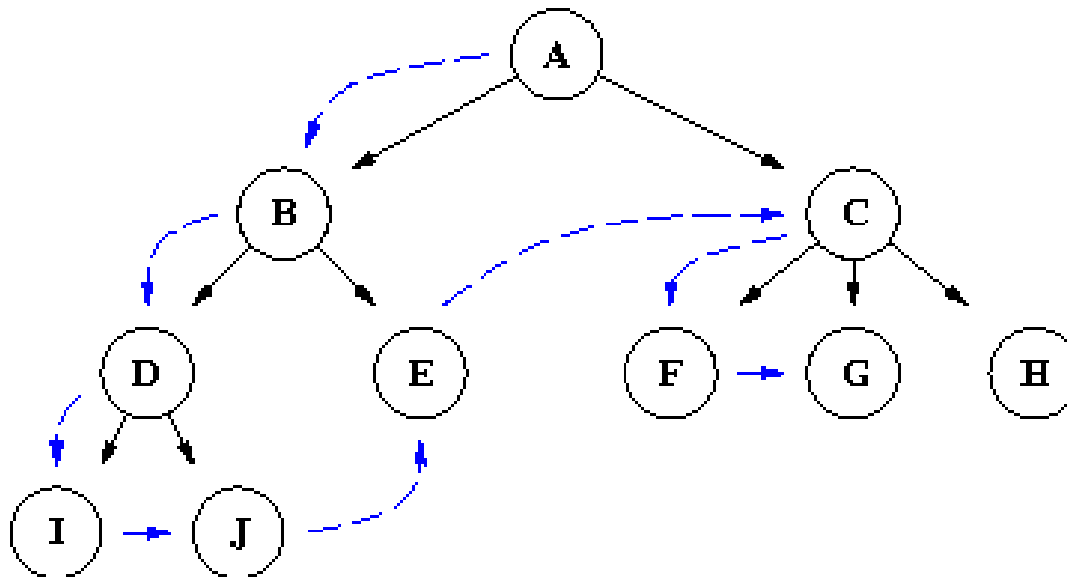


Figure 3.13 A route-finding problem. (a) The state space, showing the cost for each operator. (b) Progression of the search. Each node is labelled with $g(n)$. At the next step, the goal node with $g = 10$ will be selected.

Traženje po dubini

- Depth-first search
- Magacin kao pomoćna struktura
- Čvorovi se obilaze s leva udesno
- Implementacija: **magacin(LIFO)**



Traženje po dubini (2)

- Ne generišu se oni sledbenici koji se već pojavljuju u putu od korena do tekućeg čvora (da bi se izbegle petlje)
- Mali zahtevi za memorijom
- Ako postoje veoma dugački (ili neograničeni) putevi, ne garantuje rešenje
- Nije kompletno, ni optimalno

Traženje po dubini (3)

Prednosti:

- Uvek nalazi cilj (rešenje)
- Mali zahtevi za memorijom

Nedostaci:

- Često može biti spor
- Ako postoje veoma dugacki (ili neograničeni) putevi, ne garantuje rešenje
- Nije kompletno, ni optimalno

Primer traženja po dubini

Stanje magacina

S

S→A, S→D

S→A→B, S→A→D, S→D (S→A→S petlja – eliminiše se)

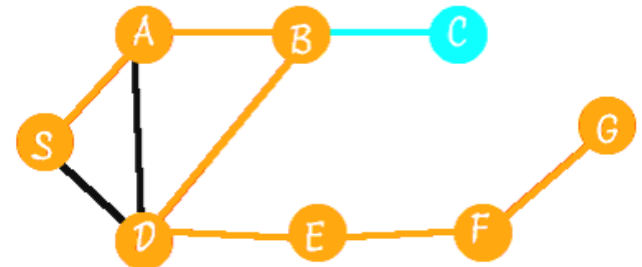
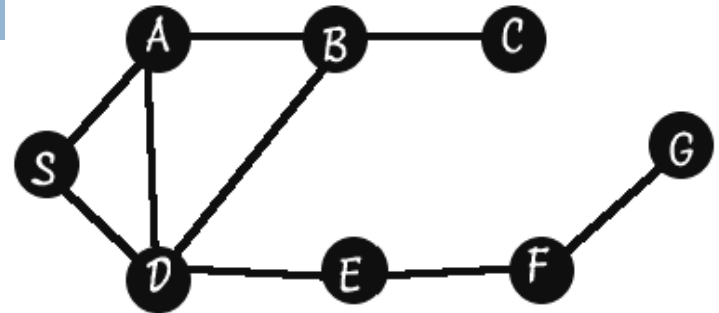
S→A→B→A (loop) **S→A→B→C**, S→A→B→D, S→A→D,
S→D

S→A→B→D, S→A→D, S→D

S→A→B→D→E, S→A→D, S→D (tri petlje: S→A→B→D→
→S, S→A→B→D→A, S→A→B→D→B)

S→A→B→D→E→F, S→A→D, S→D

S→A→B→D→E→F→G, S→A→D, S→D



Ostali algoritmi

□ Traženje ograniceno po dubini (IDS)

traženje po dubini, sa zadatom maksimalnom dubinom

□ Bidirekciono traženje

Istovremeno se polazi

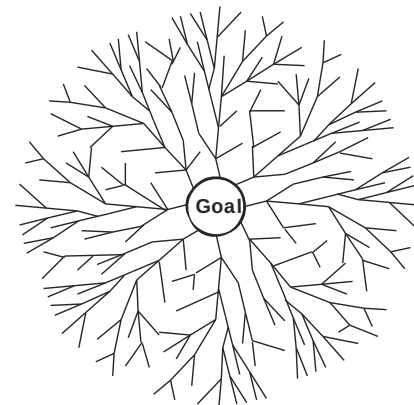
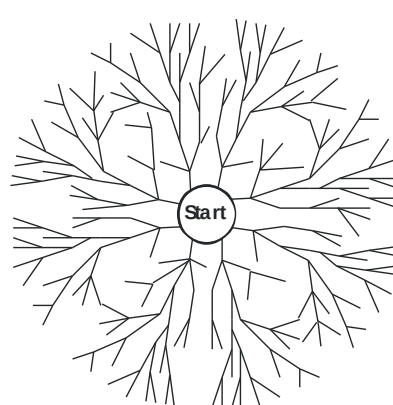
od S i od G

□ Nedeterminističko traženje

Varijanta BFS i/ili DFS,

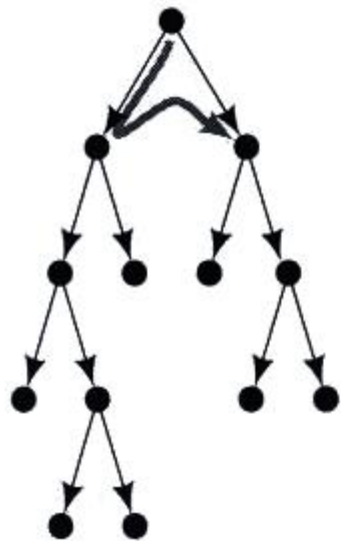
sa slučajnim izborom čvora koji se razvija

Pokušaj rešavanja problema velikog faktora grananja i mnogo nivoa

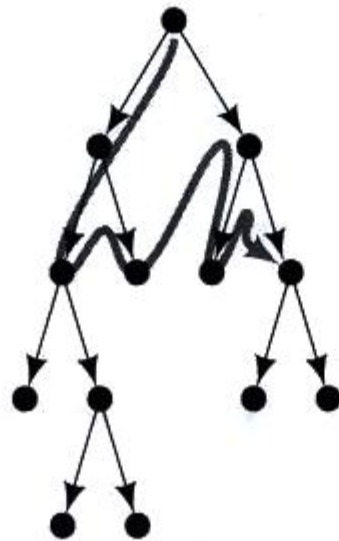


Primer IDS

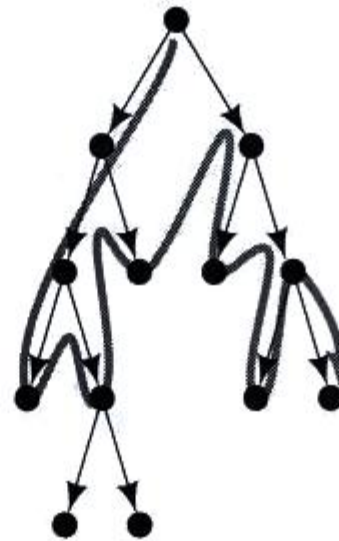
35



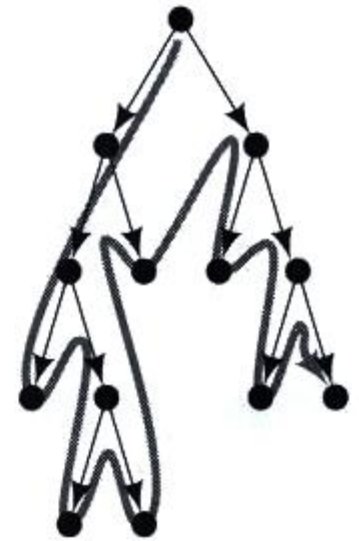
Depth bound = 1



Depth bound = 2



Depth bound = 3



Depth bound = 4

Stages in Iterative-Deepening Search

Bi-Direkcono traženje

36

Složenost: vreme i memorija

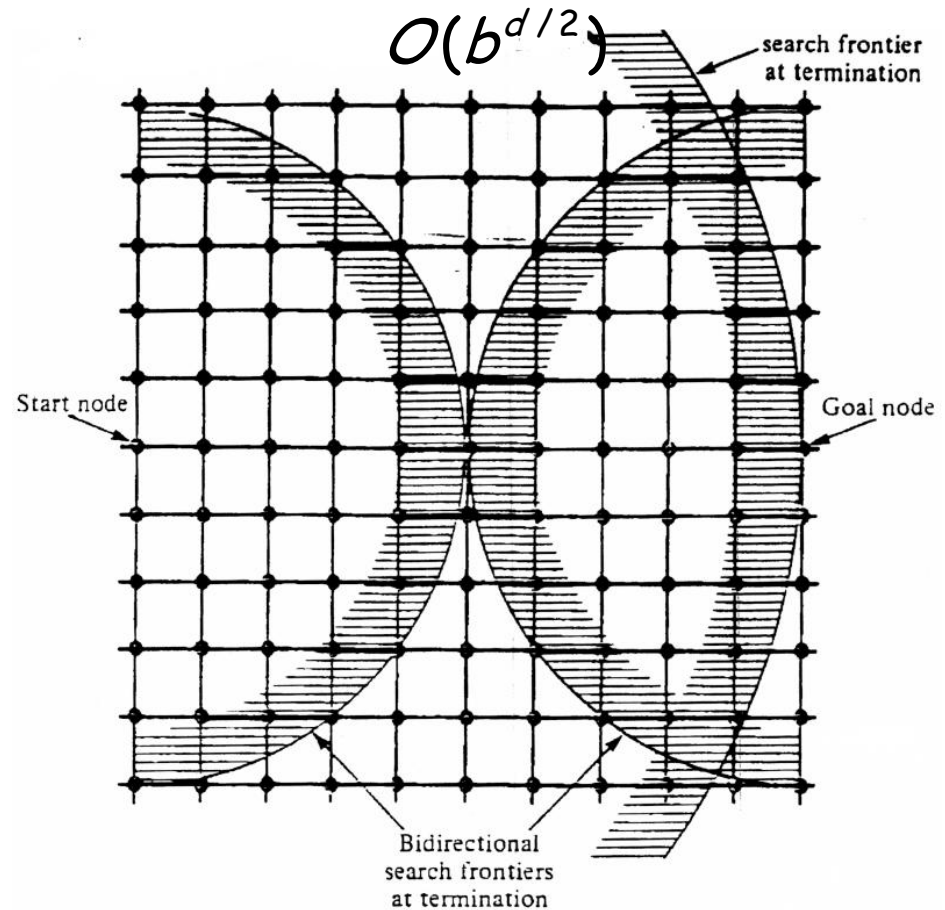
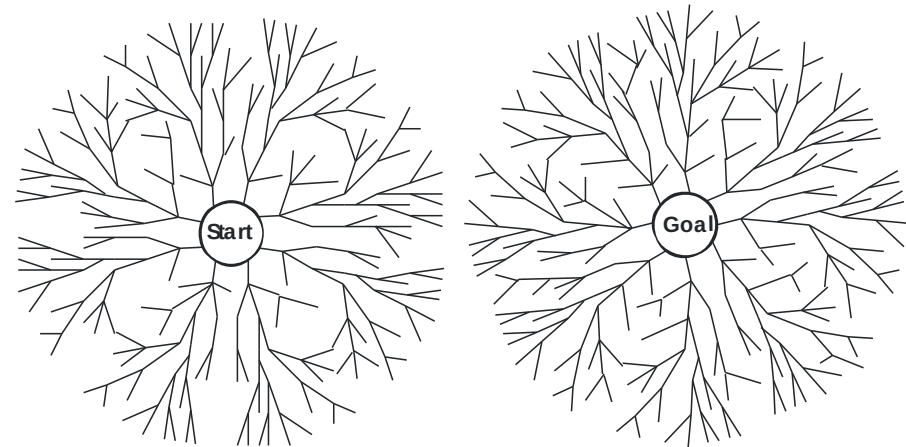


Fig. 2.10 Bidirectional and unidirectional breadth-first searches.

Kompleksnost neinformisanih algoritama

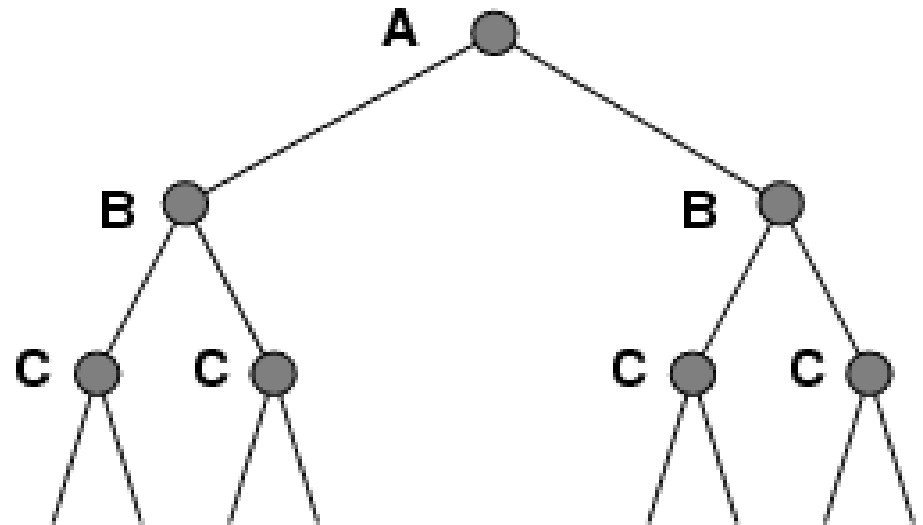
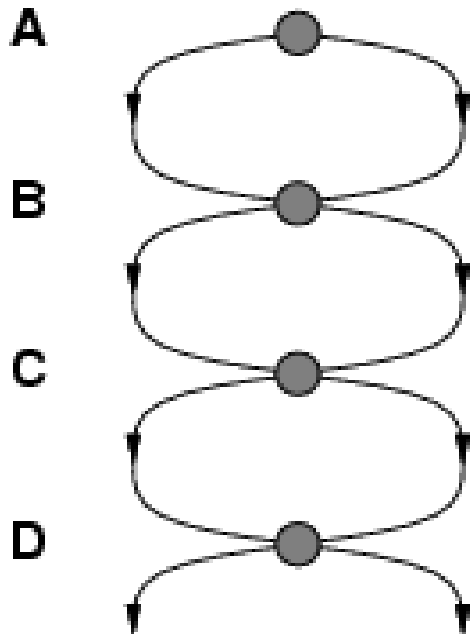
37

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes	Yes	No	No	Yes
Time	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal?	Yes	Yes	No	No	Yes

Stanja koja se ponavljaju

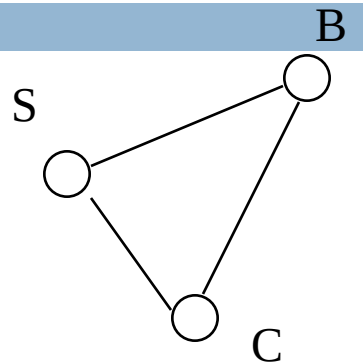
38

- Problem detekcije stanja koji se ponavljaju mogu da pretvore linearni problem u eksponencijalni!

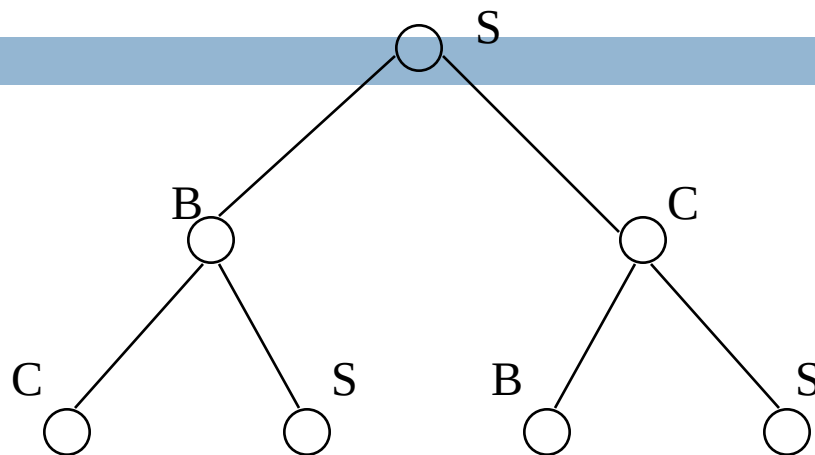


Izbegavanje stanja koja se ponavljaju

39



Prostor traženja



Primer stabla traženja

- Metoda 1 ← **Ne-optimala ali praktična**
 - ▣ Ne kreiraj put koji sadrži petlje
- Metoda 2 ← **Optimalna ali memorijski neefikasna**
 - ▣ Ne generiši stanje koje je već generisano
 - Moraju se znati sva moguća stanja (memorija!)
 - Primer: 8-puzzle problem, postoji $9! = 362,880$ stanja

Informisano (heurističko) traženje

Heuristika

- $h(n)$ – procenjena cena puta od tekućeg čvora do cilja
- $h(G)=0$
- $h(n)>0$ za ostale čvorove

Omogućava da se izabere čvor koji više obećava od ostalih

- **Prihvatljiva heuristika:** predviđena cena nikada nije veća od stvarne

Primeri heuristickih funkcija za 8-puzzle

- $h1$ = broj pločica na pogresnom mestu
- $h2$ = suma razdaljina svih pločica od njihovih krajnjih pozicija
(city block distance, Manhattan distance)

$h2(n) \geq h1(n)$ zato je $h2(n)$ bolja heuristika

Metod planinarenja

- **Hill-Climbing search**
- uvek ide prema cilju
- koristi heuristiku za nalaženje pravca koji vodi najbrže (najbliže) cilju
- analogija sa planinarenjem, noću, ako je logor na vrhu brda.



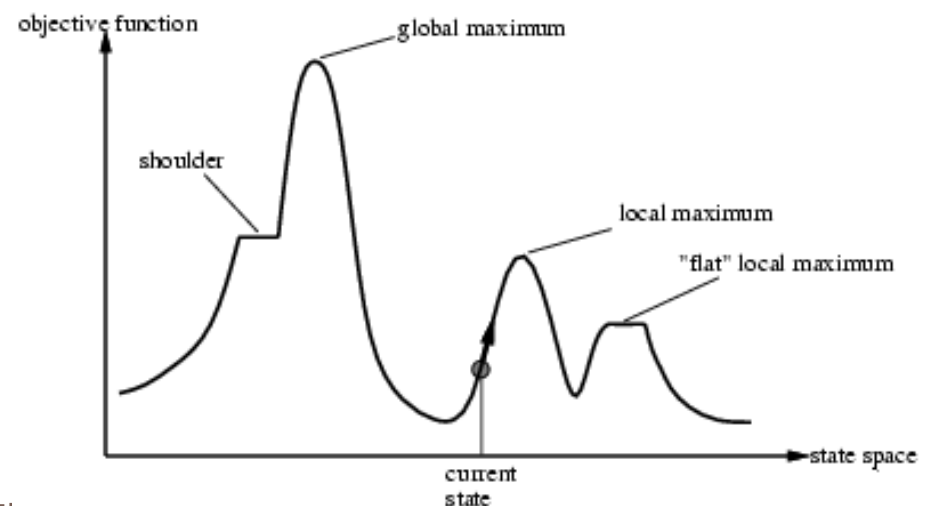
Metod planinarenja (2)

Prednosti:

- Smanjuje broj čvorova koje treba obići
- Daje efikasnije ili isto rešenje kao DFS

Nedostaci:

- "Lažni vrhovi " u kojima dolazi do izrazitih povrata
- Problem ako svi čvorovi izgledaju kao podjednako dobri
- Neophodna heuristika
- Može se desiti da se ne nađe rešenje iako ono postoji



Implementacija Hill-climbing

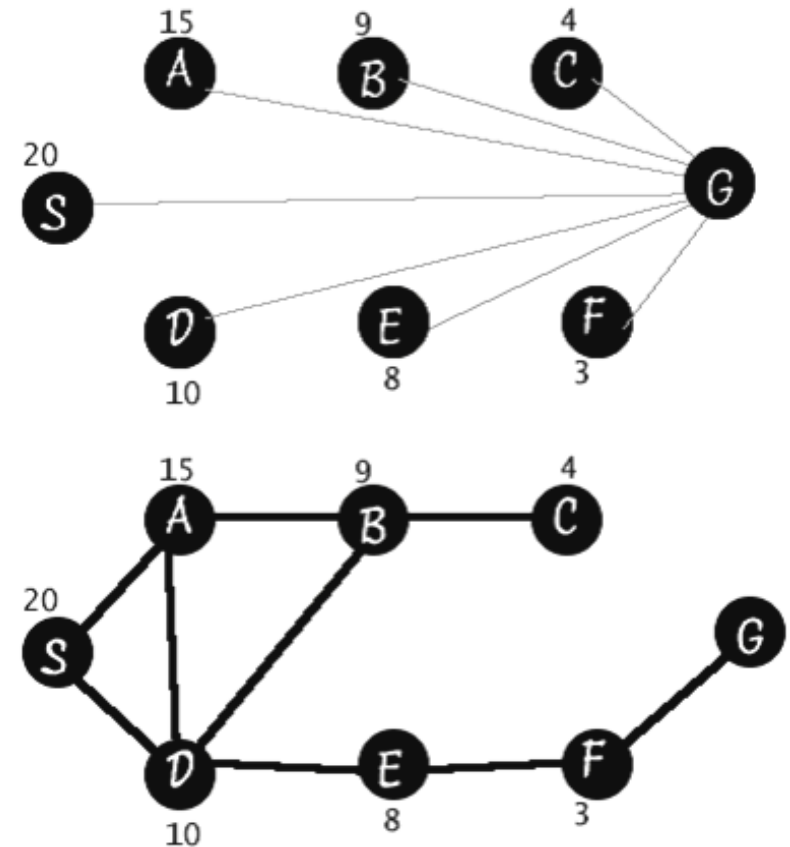
Napr. DFS + rastojananja svakog čvora do cilja

- Formirati listu čvorova koja inicijalno sadrži samo startni čvor.
- Dok se lista čvorova ne isprazni ili se ne dođe do ciljnog čvora, proveriti da li je prvi element liste ciljni čvor
 - ▣ Ako je prvi element liste ciljni čvor, ne raditi ništa.
 - ▣ Ako prvi element liste nije ciljni čvor, ukloniti ga iz liste i sortirati njegove sledbenike iz stabla pretrage (ako ih ima i ako nisu već posećeni) po rastućim vrednostima heurističke funkcije čvora (rastojanje od ciljnog čvora).
Zatim te sledbenike dodati na početak liste tako da prvi element liste bude sledbenik sa najmanjom vrednošću heurističke funkcije.
- Ako je pronađen ciljni čvor, pretraga je uspešno završena; u suprotnom pretraga je neuspešna.

Rad algoritma Hill-climbing

Stanje magacina

- S (rastojanje 20)
- Sledbenici: $S \rightarrow A(15)$, $S \rightarrow D(10)$ $\Rightarrow$ sortiranje i dodavanje u listu:
 $S \rightarrow D(10)$, $S \rightarrow A(15)$
- Sledbenici za D:
 $S \rightarrow D \rightarrow A(15)$, $S \rightarrow D \rightarrow B(9)$,
 $S \rightarrow D \rightarrow E(8)$
 $\Rightarrow$ sortiranje i dod.: **$S \rightarrow D \rightarrow E(8)$** ,
 $S \rightarrow D \rightarrow B(9)$, $S \rightarrow D \rightarrow A(15)$, $S \rightarrow A(15)$
- **$S \rightarrow D \rightarrow E \rightarrow F(3)$** , $S \rightarrow D \rightarrow B(9)$,
 $S \rightarrow D \rightarrow A(15)$, $S \rightarrow A(15)$
- **$S \rightarrow D \rightarrow E \rightarrow F \rightarrow G(0)$** ,
 $S \rightarrow D \rightarrow B(9)$, $S \rightarrow D \rightarrow A(15)$,
 $S \rightarrow A(15)$



Algoritam prvi najbolji

Best-first search

- Kompletan, nije optimalan generalno
- Ideja: koristiti **evaluacionu funkciju** $f(n)$ za svaki čvor
 - ▣ $f(n)$ obezbeđuje procenjenu totalnu cenu puta.
 - Čvor za ekspanziju se bira na osnovu minimalne vrednosti za $h(n)$
- **Implementacija:**

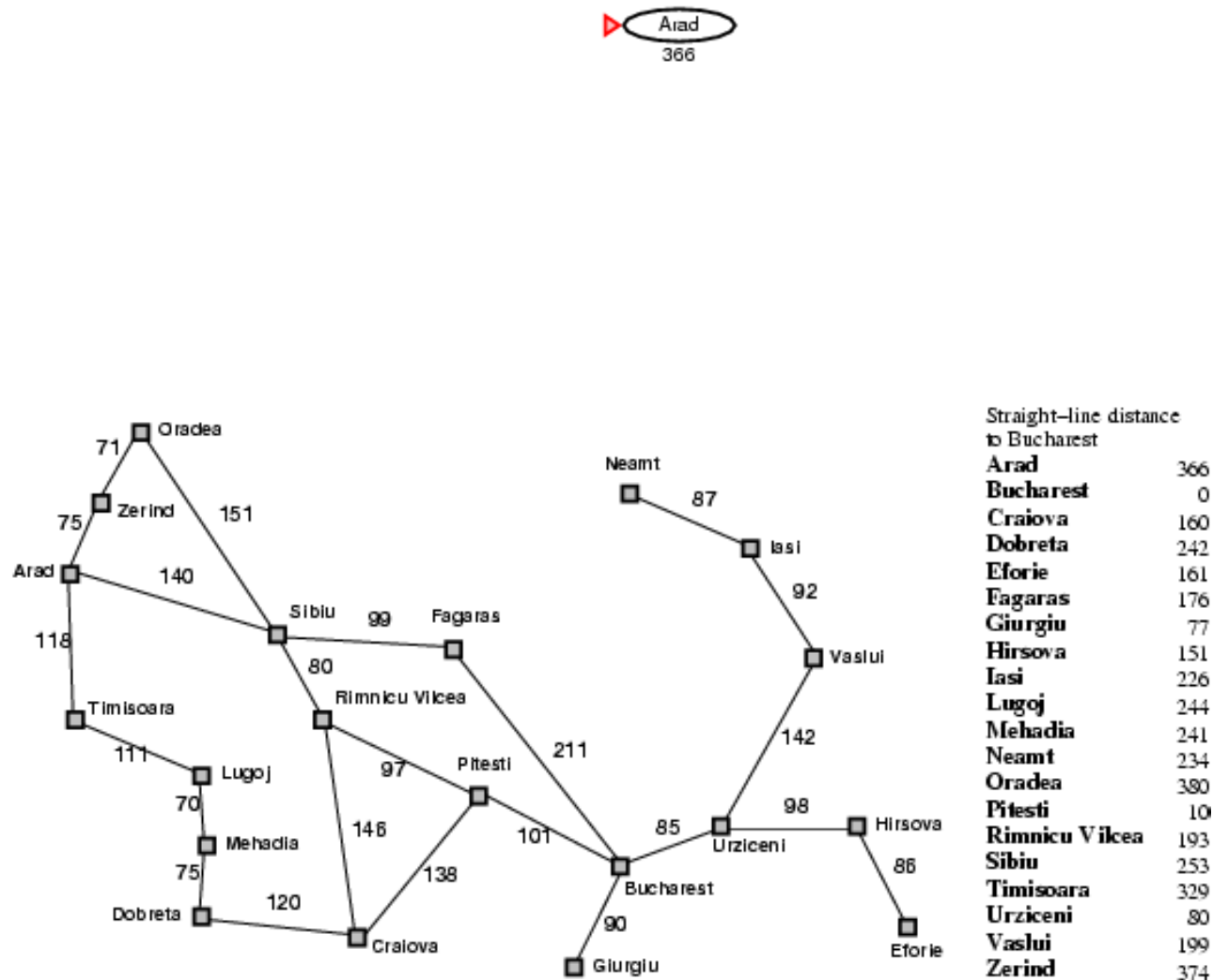
Urediti čvorove po vrednosti cene puta – **sortirani red**.
- Specijalni slučajevi:
 - ▣ Greedy best-first traženje – ekspanzija čvora koji **izgleda** da je najbliži cilju
 - ▣ A^* traženje

Implementacija Best-first search

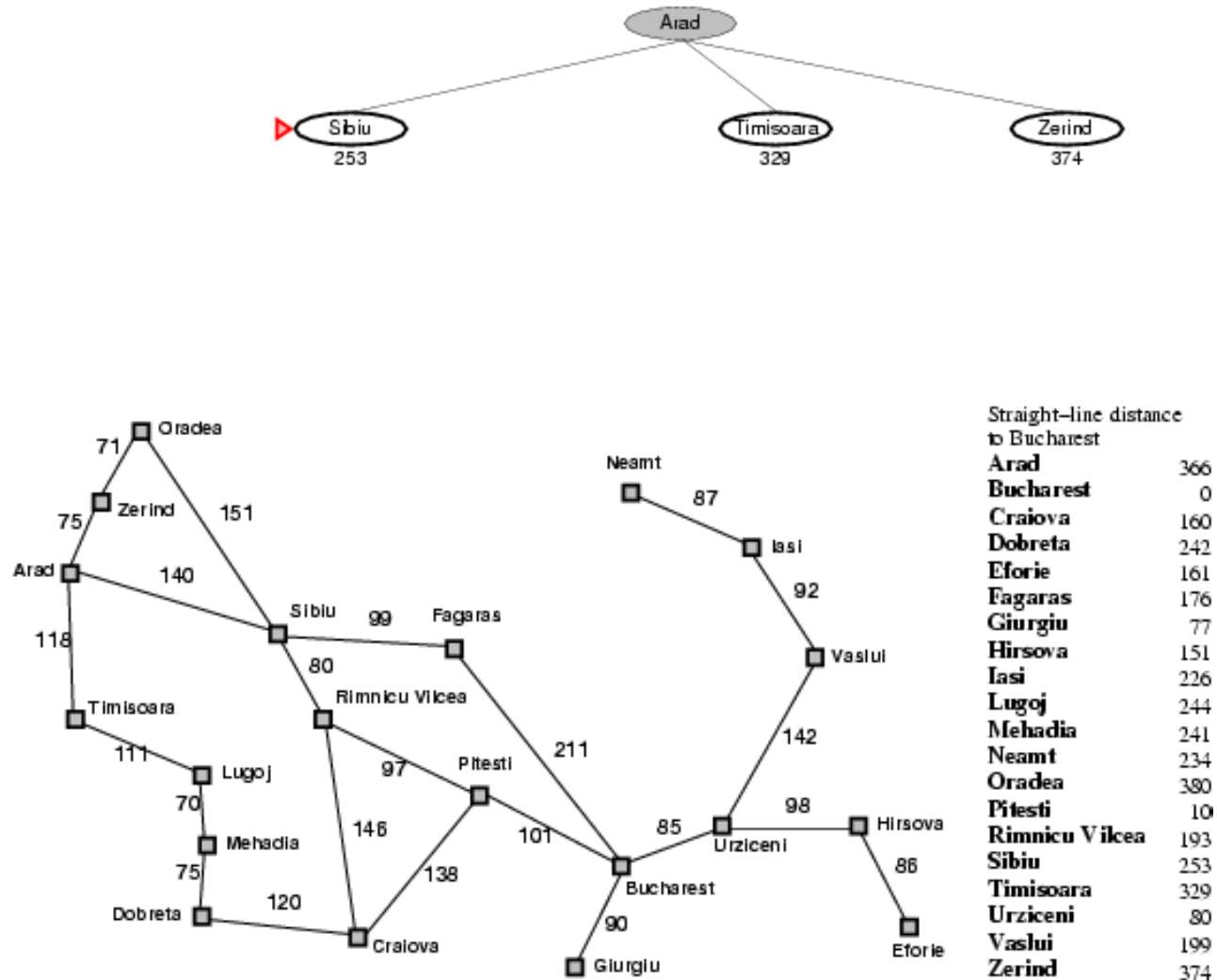
Implementacija: **sortirani red**

1. Formirati listu čvorova koja inicijalno sadrži samo startni čvor.
2. Dok se lista čvorova ne isprazni ili se ne dođe do ciljnog čvora, proveriti da li je prvi element liste ciljni čvor
 - a) Ako je prvi element liste ciljni čvor, ne raditi ništa.
 - b) Ako prvi element liste nije ciljni čvor, ukloniti ga iz liste i dodati njegove sledbenike iz stabla pretrage (ako ih ima i ako nisu već posećeni) u listu.
Celokupnu listu sortirati po rastućim vrednostima heurističkih funkcija **čvorova**.
3. Ako je pronađen ciljni čvor, pretraga je uspešno završena; u suprotnom pretraga je neuspešna.

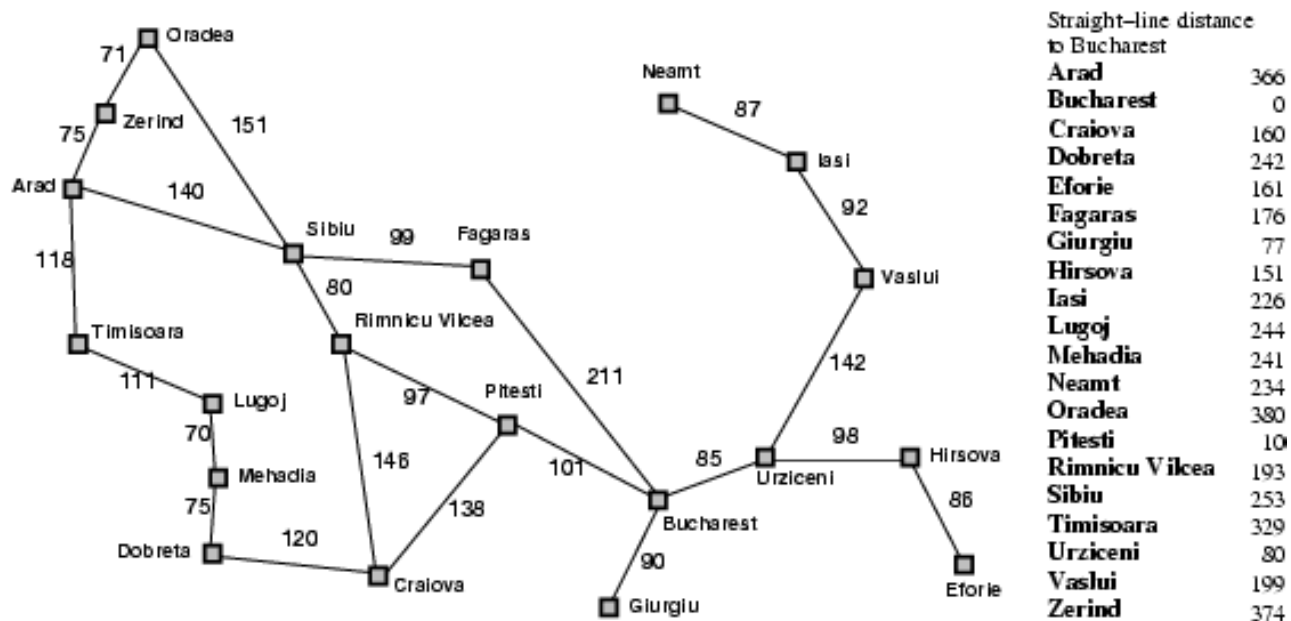
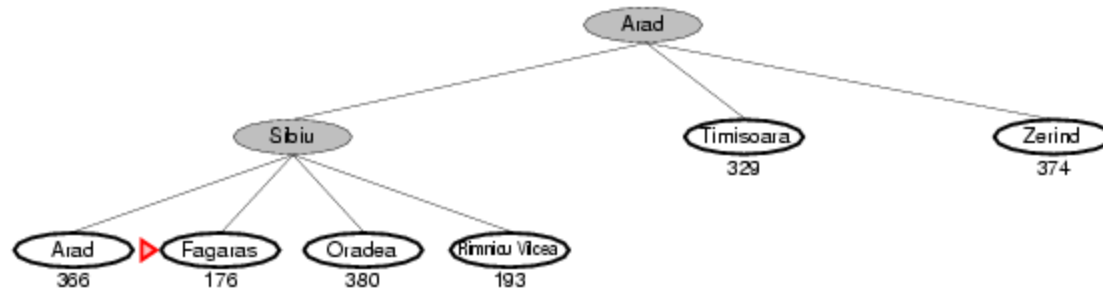
Primer: Greedy best-first search



Primer: Greedy best-first search



Primer: Greedy best-first search



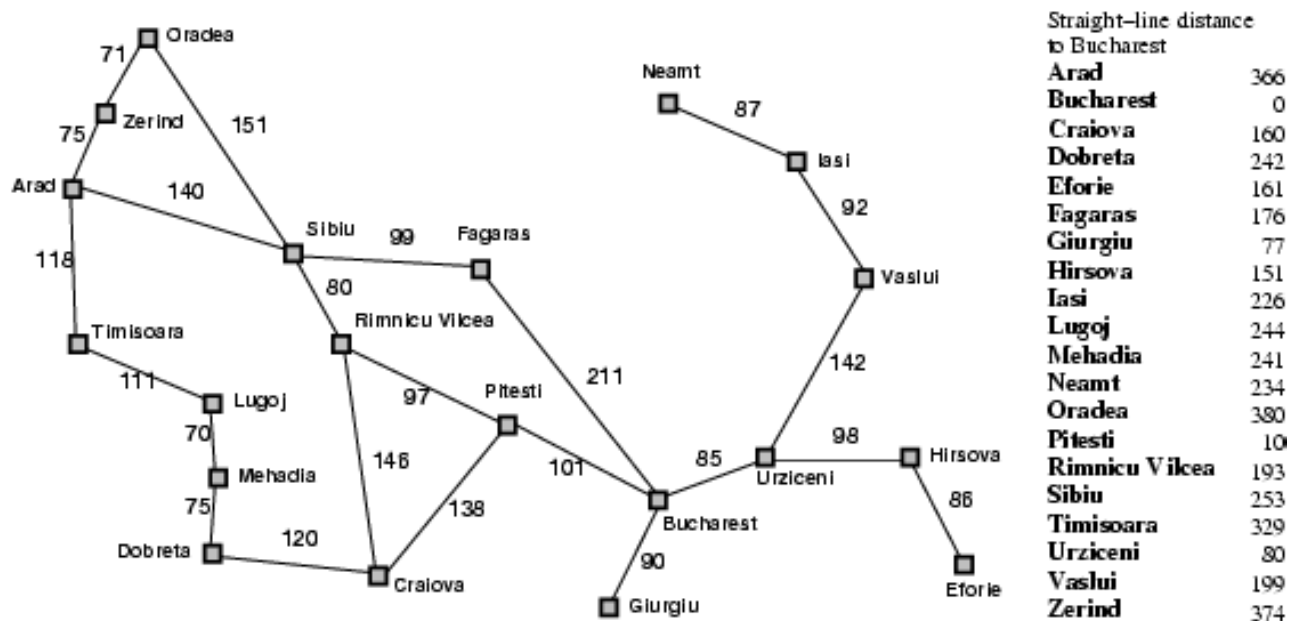
Optimalni algoritmi traženja

A* algoritam

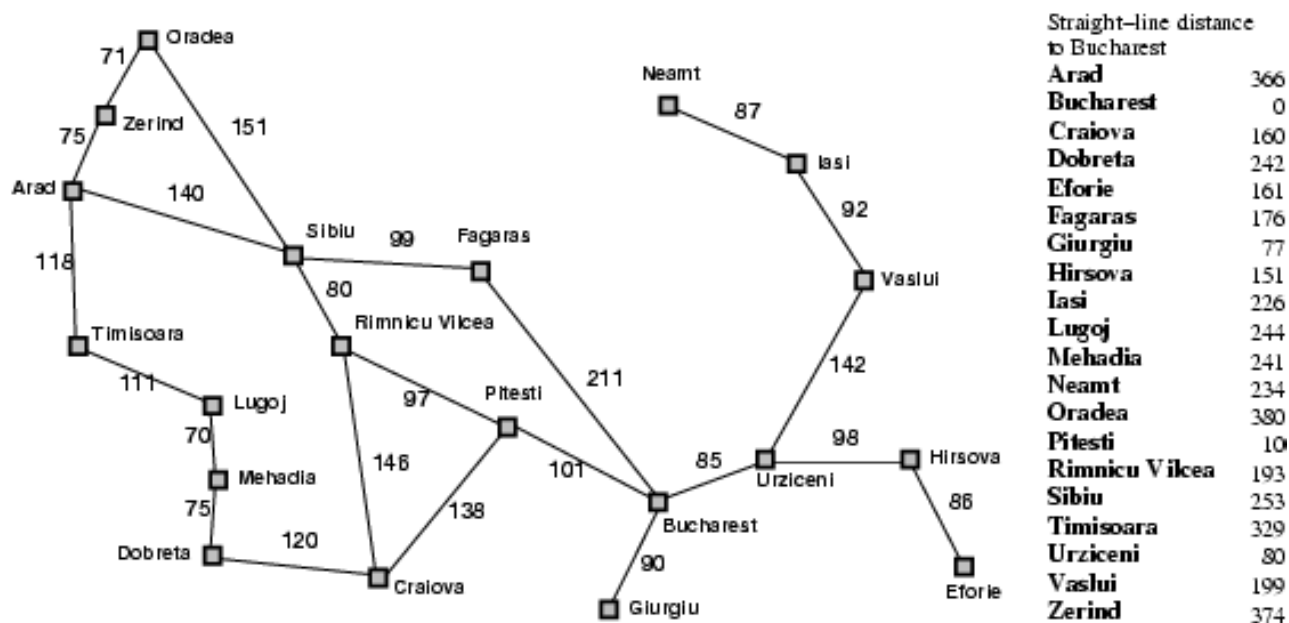
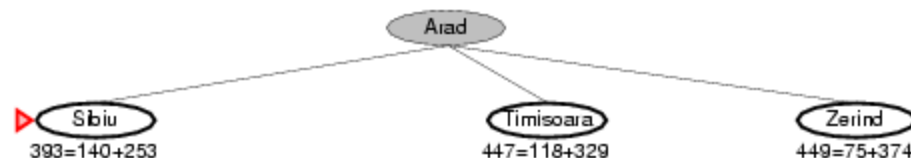
- Izbegava puteve koji su već skupi
- Kombinuje stvarnu cenu putanje $g(n)$ sa predviđenom cenom $h(n)$
- Ukupna cena $f(n)=g(n) + h(n)$
 - $g(n)$ = cena dostizanja n
 - $h(n)$ = procenjena cena od n do cilja
 - $f(n)$ = procenjena ukupna cena puta od n do cilja
 - Best First search: $f(n)=h(n)$
- **Implementacija:** Svi čvorovi u redu se **sortiraju** po vrednosti $f(n)$

Primer: A* traženje

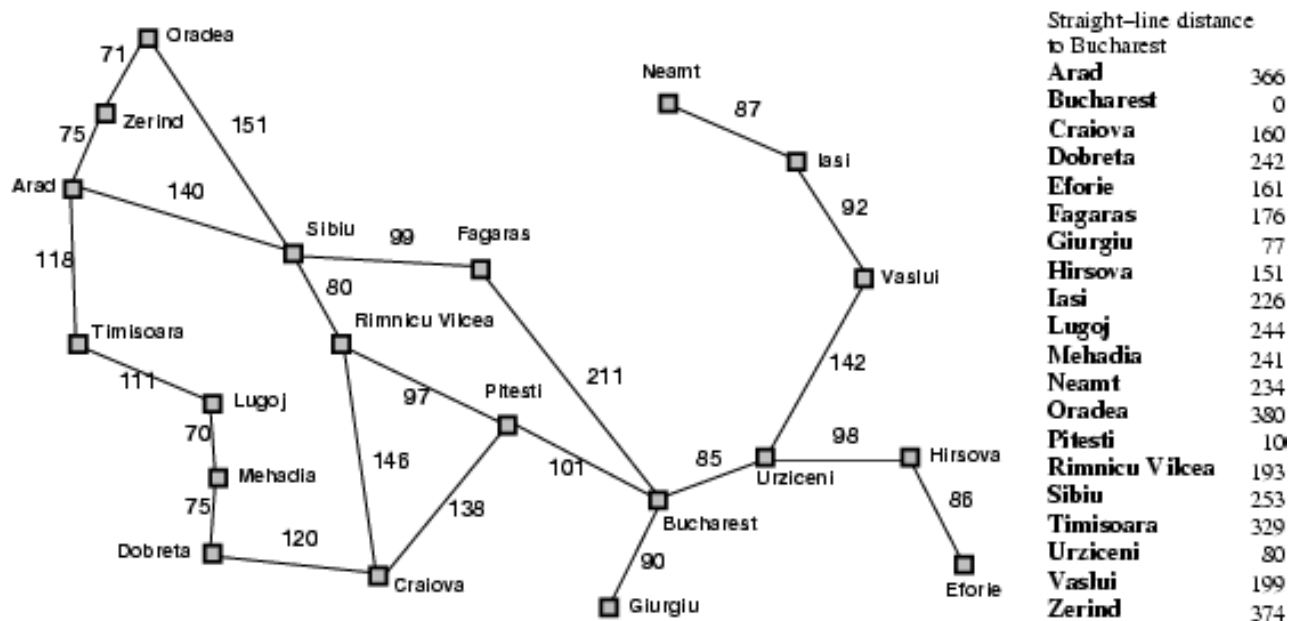
Arad
366=0+366



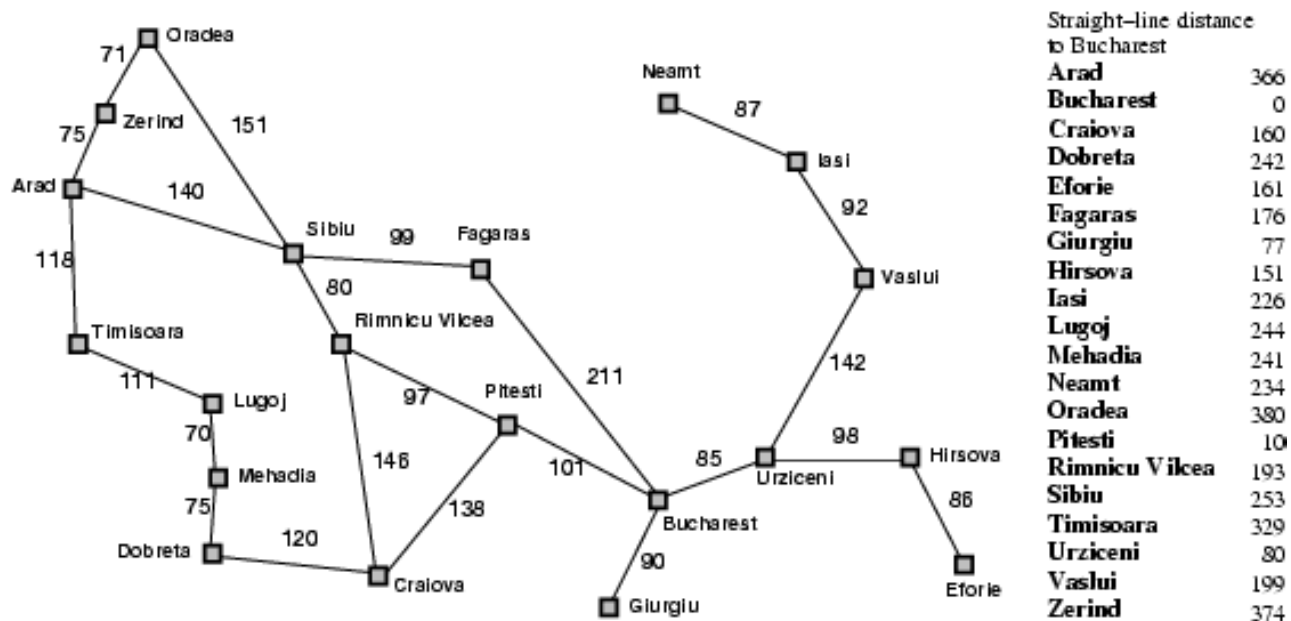
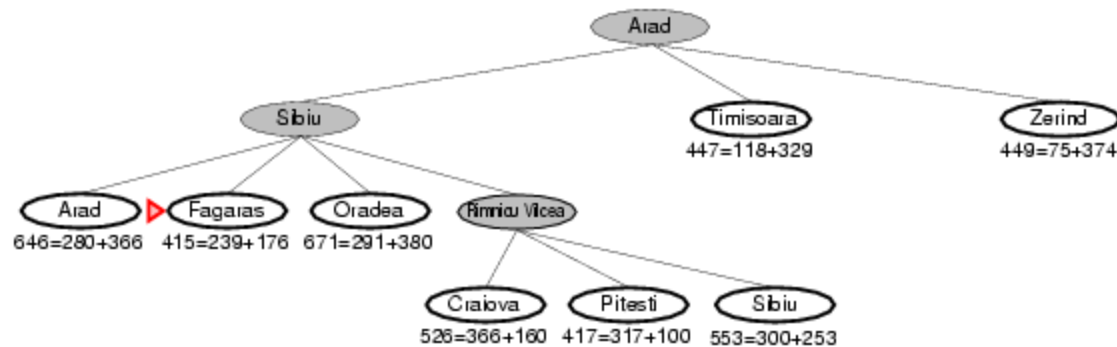
Primer: A* traženje



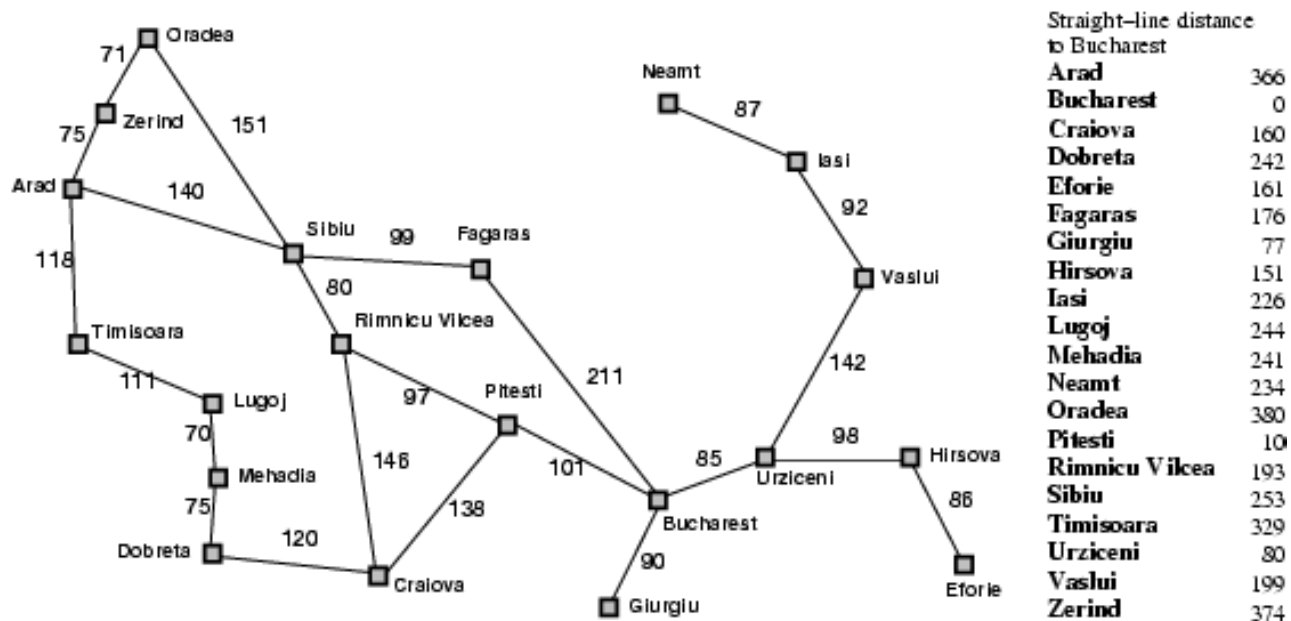
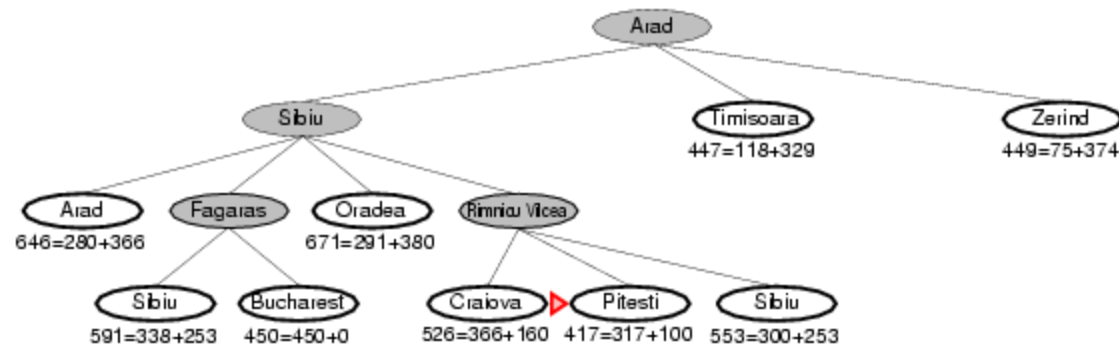
Primer: A* traženje



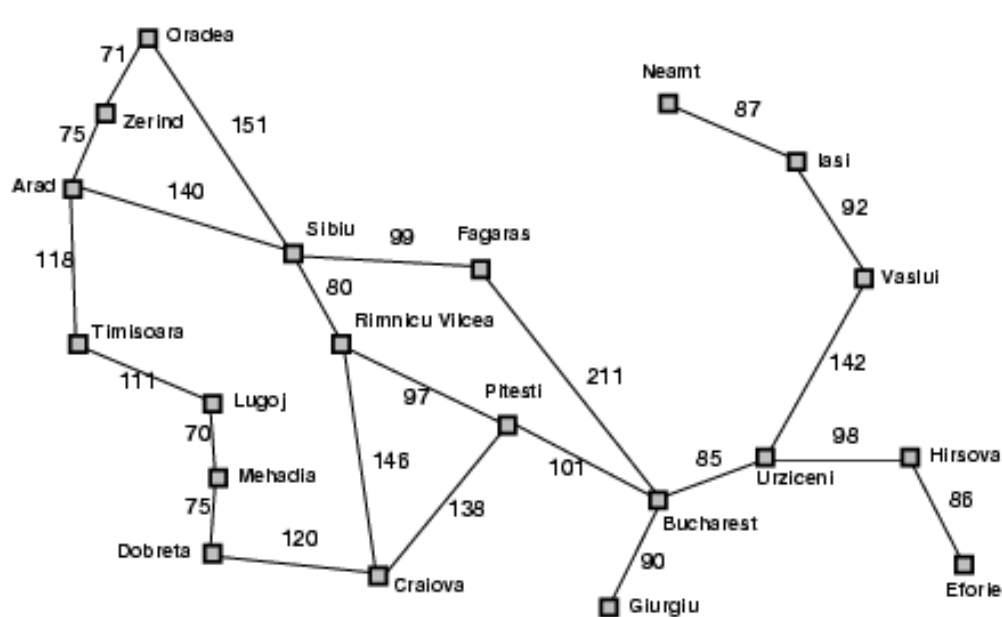
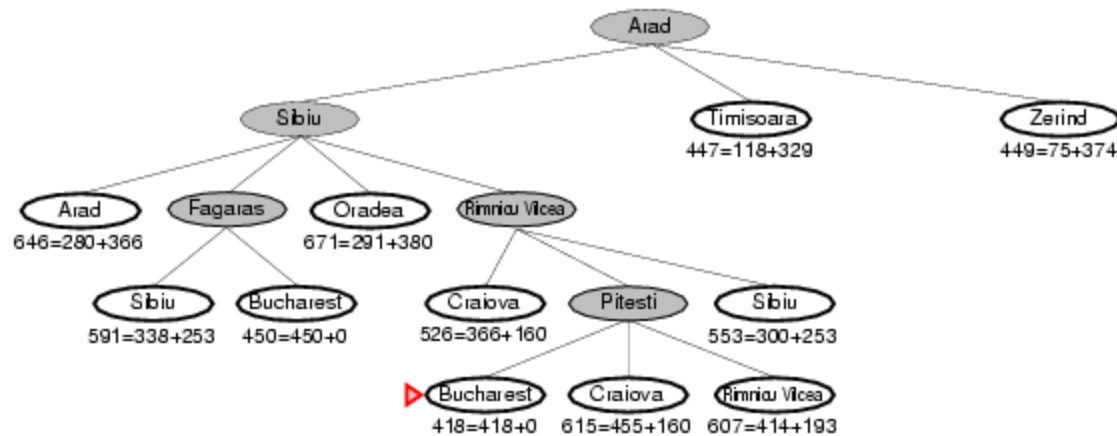
Primer: A* traženje



Primer: A* traženje



Primer: A* traženje



Straight-line distance to Bucharest

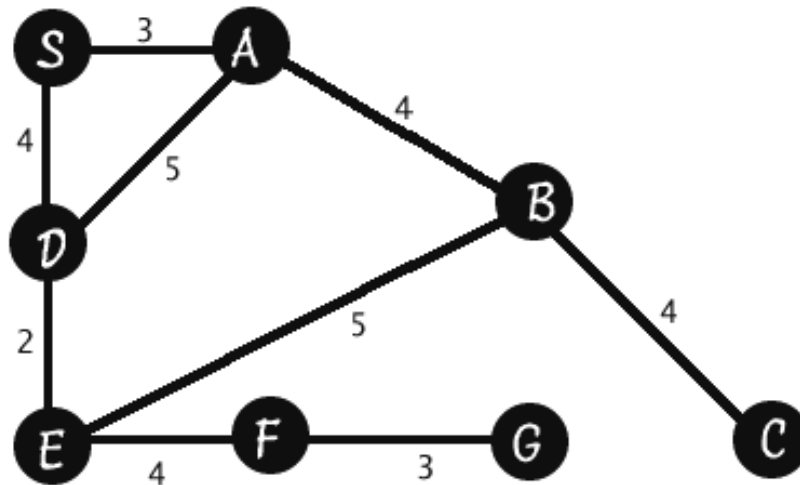
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Optimalni algoritmi traženja

Algoritam grananja i ograničavanja

□ Branch and Bound

- Implementacija: **sortiranje pomoćne strukture, stvarna cena putanje $g(n)$**



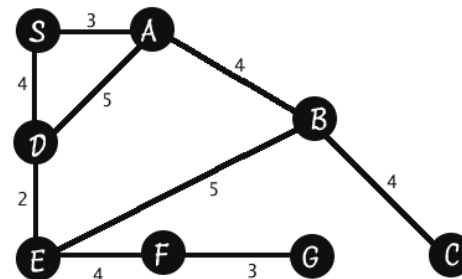
- **S** (rastojanje = 0) => **rastojanje se odnosi na dužinu puta napr do čvora, a ne na procenjeno rastojanje tog čvora do cilja** (kao kod Hill Climbing)
- Dodati: S->A (rastojanje = 3), S->D (rastojanje = 4),
Sortiranje: **S->A** (rastojanje = 3), S->D (rastojanje = 4)
- Dodati: S->A->B(**3 + 4 (rastojanje od A do B) = 7**), S->A->D(**8**),
Sortiranje: **S->D(4)**, S->A->B(**7**), S->A->D(**8**)

Rad algoritma B&B (nastavak)

- S->D->A(9), S->D->E(6), S->A->B(7), S->A->D(8),
Sortiranje: **S->D->E(6)**, S->A->B(7), S->A->D(8), S->D->A(9)
- S->D->E->B(11), S->D->E->F(10), S->A->B(7), S->A->D(8), S->D->A(9),
Sortiranje: **S->A->B(7)**, S->A->D(8), S->D->A(9), S->D->E->F(10), S->D->E->B(11)
- S->A->B->C(11), S->A->B->E(12), S->A->D(8), S->D->A(9), S->D->E->F(10), S->D->E->B(11),
Sortiranje: **S->A->D(8)**, S->D->A(9), S->D->E->F(10), S->D->E->B(11),
S->A->B->C(11), S->A->B->E(12),

Nadalje: [Najkraći put] => [lista of puteva]

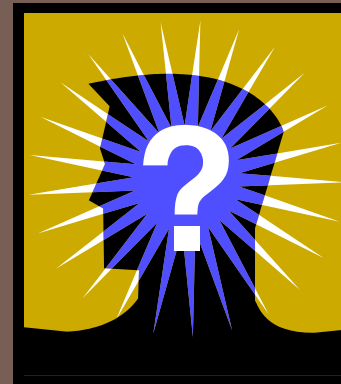
- S->A->D(8) => S->A->D->E(10)
- S->D->A(9) => S->D->A->B(13)
- S->D->E->F(10) => **S->D->E->F->G(13)** cilj je dostignut sa cenom 13!



Razvoj preostalih puteva, i traženje da li postoji neki sa cenom manjom od 13.

- S->A->D->E(10) ima manju cenu u narednom koraku, razvijanjem dobijamo:
- S->A->D->E->B(15), S->A->D->E->F(14), veći su od 13, ignorišemo ih, **itd.**

PITANJA?



Dileme?

Komentari?

